Reviewer Pack — Minimal Rank of Tropicalized K-theory over Resolution Proof …

Entry 1b66be60747e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 13:58:10 UTC

Minimal Rank of Tropicalized K-theory over Resolution Proof Trees

Entry ID: `1b66be60747e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 13:58:10 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (K-theory) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For any d-dimensional pseudoexpectation on a Boolean function, the minimal rank of its tropicalization is upper-bounded by the width of the corresponding resolution proof tree.', 'Formally, for a CNF F with m clauses and n variables, let P(F) be the resolution proof tree associated with F. Then, for any d-dimensional pseudoexpectation E on F, τ(E) has minimal rank at most d * log(n + m) / log(log(n + m)).', 'If this statement is false, there exists a CNF F with m clauses and n variables such that the resolution proof tree width of F exceeds d * log(n + m) / log(log(n + m)).']

Rationale (proposer's reasoning):

> ['Tropicalization provides a bridge between geometric structures in tropical geometry and logical structures in complexity theory.', 'K-theory is an algebraic invariant that measures the complexity of certain types of vector bundles, and its tropicalization has been used to study quantum information theory.', 'The conjecture suggests that resolution proof trees, which are fundamental objects in the study of SAT solvers, can be related to the geometric structure captured by K-theory.']

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `cfece74137d850ae`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated CNFs with n variables and m clauses, the minimal rank of the tropicalization τ(E) does not exceed d * log(n + m) / log(log(n + m)) for any pseudoexpectation E. The conjecture is falsified if there exists at least one CNF such that the resolution proof tree width exceeds this bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropicalization K-theory" AND "resolution proof complexity" - "minimal rank tropical geometry" AND "resolution proof tree" - "pseudoexpectation Boolean function" AND resolution proof tree

Top relevant hits considered: - [s2:ca44ef9be97a1c211295785ba4b0290db90a6bf2] international workshop on functions in algebra and geometry 2nd-6th 2022

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(tuple(sorted(clause)))
        return tuple(cnf)

    def resolution_width(cnf):
        clauses = list(cnf)
        while True:
            new_clauses = []
            for i in range(len(clauses)):
                for j in range(i + 1, len(clauses)):
                    clause1, clause2 = clauses[i], clauses[j]
                    if any(abs(x) == abs(y) for x in clause1 for y in clause2):
                        new_clause = tuple(sorted([x for x in clause1 + clause2 if x != -x[0]]))
                        if new_clause not in new_clauses:
                            new_clauses.append(new_clause)
            if not new_clauses:
                break
```

```
                clauses.extend(new_clauses)
            return len(clauses)

        def pseudoexpectation(cnf):
            n = max(abs(x) for clause in cnf for x in clause)
            return sum(1 / (n + 1) ** len(clause) for clause in cnf)

        def tropicalization(pseudoexp):
            return pseudoexp

        d = random.randint(1, 3)
        n = random.randint(5, 20)
        m = random.randint(n, 2 * n)
        cnf = generate_cnf(n, m)

        width = resolution_width(cnf)
        pseudoexp = pseudoexpectation(cnf)
        tau_e = tropicalization(pseudoexp)
        rank = len(tau_e)  # Assuming the rank is simply the length of the tropicalized vector

        return {
            "metric_name": "minimal_rank",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": width <= d * math.log(n + m) / math.log(math.log(n + m)),
            "counterexample": "" if width <= d * math.log(n + m) / math.log(math.log(n + m)) else f"wi
        }

    if __name__ == "__main__":
        import sys
        seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [random.getrandbits(32) for _ in r

        results = []
        for seed in seeds:
            result = run_trial(seed)
            print(f"TRIAL: {result}")
            results.append(result)

        mean_value = sum(res["metric_value"] for res in results) / len(results)
        support_fraction = sum(int(res["conjecture_holds"]) for res in results) / len(results)

        if all(res["conjecture_holds"] for res in results):
            print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
        elif support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
        else:
            first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
            print(f"RESULT: FALSIFIED counterexample=\"width exceeds bound\" first_failing_seed={first
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1d4bd9d.py", line 75, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1d4bd9d.py", line 56, in run_trial
    width = resolution_width(cnf)
            ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1d4bd9d.py", line 36, in resolution_wi
    new_clause = tuple(sorted([x for x in clause1 + clause2 if x != -x[0]]))
                                                                   ~^^^
TypeError: 'int' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test code and ensure that it completes without crashing. If the crash is resolved, re-evaluate the results according to the pre-registered support and falsification conditions.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12889
2	preregistration	ollama_remote	glm4:latest	0	0	6141
3	novelty	ollama_remote	glm4:latest	0	0	4539
4	novelty	ollama_remote	glm4:latest	0	0	5688
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13297
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11464
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10000
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9259
9	judge	ollama_remote	glm4:latest	0	0	13923

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 87200 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1b66be60747e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1b66be60747e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1b66be60747e.tar.gz` (if generated)